

TransitDuet: bi-level reinforcement learning for dynamic bus timetable optimisation and holding control

Abstract

Bus timetable planning and real-time holding control are operationally coupled yet typically addressed in isolation. This paper presents TransitDuet, a bi-level reinforcement learning framework that jointly optimises rolling dispatch timetables and station-level holding decisions under stochastic demand and traffic conditions. The upper policy selects discrete planned-headway adjustments that define a dynamic rolling timetable, while the lower policy, a robust ensemble soft actor-critic controller with Lagrangian headway constraints, executes this timetable through station holding. Cross-level coordination operates through a dynamic timetable channel, holding-feedback state augmentation, and per-dispatch hindsight credit assignment, with fleet-size feasibility maintained by measurement-driven online gradient descent penalty adaptation. Evaluated in a calibrated bidirectional bus-corridor microsimulation with elastic fleet budgets across ten random seeds, TransitDuet achieves a mean composite cost of 1.634, which is 3.2% lower than a target-headway-only variant (1.687) and 11.5% lower than the best fixed-timetable baseline (1.846). Mean total passenger time is 18.15 minutes, compared with 19.30 and 19.92 minutes for these respective comparators. These results demonstrate that jointly optimising dispatch planning and holding control yields measurable performance gains over single-level holding controllers and decoupled configurations on the evaluated corridor.

Keywords: bus timetable optimisation; bus holding control; hierarchical reinforcement learning; reinforcement learning; constrained reinforcement learning

1 Introduction

High-frequency bus systems face a persistent coordination challenge. Timetables are planned offline to balance fleet utilisation against passenger demand, yet the headways they prescribe degrade within minutes once buses encounter stochastic traffic and boarding delays. Real-time holding control can restore regularity, but only within the headway structure it inherits. When the timetable itself is poorly matched to demand, dispatching too few buses in the peak or too many in the off-peak, holding alone may not recover acceptable service. Planning and control are thus coupled: the quality of one layer bounds the achievable performance of the other.

Despite this coupling, existing work treats the two problems separately. The timetable optimisation literature formulates headway or frequency design as mixed-integer programmes solved offline [1], while the holding control literature trains reinforcement learning (RL) agents that take the timetable as given [2, 3]. Joint optimisation would seem a natural application of hierarchical reinforcement learning (HRL), yet three gaps prevent a straightforward application.

Asynchrony between planning and control.

HRL frameworks such as the options framework [4] and goal-conditioned methods such as HIRO [5] assume that the upper level sets subgoals or options that can be evaluated through a lower-level execution segment. In bus operations, the upper policy acts at dispatch events (once per bus departure), while the lower policy acts at every station arrival across all active buses. The two levels observe different state spaces: the upper level sees route-level demand forecasts and fleet status, whereas the lower level sees per-bus headway deviations and passenger loads. They also interact through delayed feedback spanning an entire trip lifecycle. More importantly, several dispatch goals are active concurrently: one upper decision may set the planned launch time of a future trip while

buses dispatched under earlier goals continue to move and hold at stations. Thus the issue is not only the absence of a shared clock, but also the need to optimise and credit overlapping timetable edits.

Cross-advantage under asynchrony.

Pan et al. [6] recently introduced cross-advantage augmentation for bi-level robot locomotion, where an upper style policy and a lower tracking policy execute synchronously on the same state. Their augmented advantage $\tilde{A}_U = A_U + \beta A_L$ directly sums the two advantages at each time step. This mechanism is not directly applicable when the two levels operate on different timescales with different state and action spaces: no single time step jointly defines A_U and A_L . Generalising cross-level coupling to this asynchronous, multi-timescale setting requires propagating execution outcomes over the variable-length trip lifecycle induced by each dispatch. It also requires exposing lower-level holding statistics to the upper policy through state augmentation rather than reward backflow.

Fleet and regularity constraints.

Bus operations also impose feasibility-style constraints, including a fleet-size budget N_{fleet} and bounded headway deviation. Fixed-coefficient reward shaping may not accommodate these constraints reliably. Constrained RL methods such as reward constrained policy optimisation (RCPO) [7] address single-level constraint satisfaction but do not account for constraints that span a bi-level structure, where the upper level controls resource allocation and the lower level controls service regularity.

We propose TransitDuet, a bi-level reinforcement learning framework designed to address these three gaps. The upper policy π_U selects a discrete planned-headway adjustment for the next same-direction trip; the simulator converts the resulting planned headway into a planned launch time, so the upper directly modifies the rolling timetable. The lower policy π_L is a soft actor-critic (SAC) with Lagrangian penalties [8] that selects holding times at each station arrival, with parameters shared across buses. Two auxiliary upper-only signals—a holding-feedback state channel and a per-dispatch hindsight credit—provide cross-level credit assignment without injecting upper signals into the lower transitions. Fleet-size feasibility is maintained through θ -weighted online gradient descent on a measurement vector, inspired by the ApproPO framework [9], while headway uniformity enters the lower-level objective through Lagrangian relaxation.

Contributions.

The specific contributions of this paper are:

1. Dynamic timetable–holding formulation. We cast rolling bus timetable optimisation and holding control as a bi-level Markov decision process with heterogeneous state-action spaces and asynchronous transitions.
2. Discrete timetable and holding policies. The upper policy chooses from a finite dispatch-headway adjustment grid, and the lower policy chooses from a finite holding-time grid augmented with adjacent planned-dispatch context. This discretisation matches operational command granularity and isolates the value of dynamic timetable decisions against fixed-headway baselines.
3. Adaptive constraint handling via θ -weighted online gradient descent (OGD) and Lagrangian relaxation. At the upper level, measurement-driven OGD adjusts the reward weight on fleet overshoot. At the lower level, a dual variable penalises headway deviation. OGD removes the need to hand-tune an overshoot coefficient but provides no worst-case feasibility guarantee.
4. Empirical evaluation on a calibrated corridor with ten random seeds, reporting both service-quality metrics (waiting time, headway CV, total passenger time) and timetable-execution diagnostics (planned-headway variability, dispatch lateness).

Results preview.

We evaluate on a high-frequency bus corridor with 22 stations, 264 daily trips, demand stochasticity $\sigma_{\text{demand}} = 0.15$, and elastic fleet $N_{\text{fleet}} \in [8, 16]$ sampled per episode. All learned methods use

the same simulator and random-seed protocol, with the target-headway-only model reported as an ablation rather than as the main TransitDuet configuration.

2 Related work

2.1 Reinforcement learning for bus holding control

Research on real-time bus holding spans analytical control rules and learned policies. Early analytical rules remain important reference points. Daganzo [10] derived a headway-based control law for eliminating bunching. Xuan et al. [11] proposed dynamic holding strategies that balance schedule reliability and regular headways through optimal linear-control analysis. We include corresponding classical baselines to test whether learning adds value beyond interpretable holding rules.

RL studies learn holding decisions directly from simulated operations. Alesiani and Gkiotsalitis [12] formulated high-frequency bus holding as a value-based RL problem. Wang and Sun [2] introduced a multi-agent deep RL controller in which each bus acts as an agent under shared training. Zhang et al. [3] developed a single-agent robust deep RL framework for bidirectional timetabled bus lines, showing that categorical identifiers help a shared policy operate across heterogeneous vehicles, stations, and trips.

Subsequent work addressed asynchronous control and robustness. Wang and Sun [13] developed an asynchronous multi-agent reinforcement learning (MARL) framework with counterfactual credit assignment, while Wang and Sun [14] combined distributional MARL with meta-learning to handle service perturbations. Multi-line and multi-strategy settings include shared-corridor multi-objective MARL [15], cooperative holding with stop-skipping [16], hierarchical MARL that combines holding and speed control [17], and graph-aware deep RL for multi-line busy corridors [18].

Other studies focus on RL design choices. He et al. [19] used Q-learning with multistage look-ahead for adaptive holding. Xu et al. [20] compared state, action, and agent configurations for RL-based bunching mitigation. Yu et al. [21] used language-model-assisted reward generation for generic bus holding policies. Zhang and Zheng [22] proposed robust ensemble soft actor-critic (RE-SAC), an ensemble-critic variant of soft actor-critic for bus fleet control that separates aleatoric and epistemic risk signals. We adopt this critic structure at both levels of TransitDuet.

Many RL-based holding controllers treat the timetable, target headway, or dispatch plan as an exogenous input. They can react to realised bunching but do not use systematic execution evidence—such as repeated lateness, excessive onboard delay, or recurrent over-holding—to revise future dispatch times. TransitDuet addresses this limitation by coupling a holding controller with a higher-level rolling timetable policy that feeds execution outcomes back into dispatch planning.

2.2 Bus timetable optimisation

Timetable design has traditionally been approached through mathematical programming and meta-heuristic search. Ibarra-Rojas et al. [23] formulated a mixed-integer programme that jointly determines departure times and vehicle assignments to minimise a weighted combination of passenger waiting time and operator cost. They solved it by branch-and-bound with valid inequalities. Gkiotsalitis and Cats [1] proposed a bi-level model in which the upper level selects headways to minimise total passenger cost and the lower level solves a vehicle-scheduling problem using adaptive large neighbourhood search. Frequency-setting variants, such as the work of Cats and Glück [24], optimise the number of departures per period rather than individual departure times. They typically rely on assignment models to estimate passenger flows under each candidate frequency vector.

Many timetable formulations rely on static demand forecasts and assumed travel-time conditions. They are optimised offline against expected conditions and then handed off to operations without a feedback loop. Under traffic incidents, demand surges, or driver variability, the timetable may become infeasible, leaving holding controllers to absorb the mismatch. The combinatorial structure can also limit scalability as the number of trips and stops increases. TransitDuet instead uses a closed-loop approach in which the upper policy maps aggregate state features to planned-headway

adjustments and receives lower-level execution feedback through the cross-level coupling described in Section 4.4.

2.3 Hierarchical and multi-timescale reinforcement learning

Hierarchical reinforcement learning decomposes complex tasks across multiple temporal scales. The options framework of Sutton et al. [4] introduced temporally abstract actions, each consisting of an initiation set, an intra-option policy, and a termination condition. It enables planning over multiple timescales within a single Markov decision process (MDP). Feudal networks [25] operationalize this idea with a manager that sets subgoals in a learned latent space and a worker that executes primitive actions to reach them. HIRO [5] introduced off-policy corrections that relabel high-level goals in hindsight, making hierarchical training practical in continuous-control domains. Lee et al. [26] and Metelli et al. [27] studied multi-timescale settings more broadly, including interactions among agents operating at different decision frequencies.

These methods provide useful abstractions for temporal hierarchy, but many assume synchronous execution or a shared state space across levels. In transit operations, the timetable agent acts at dispatch events, whereas the holding agent acts at every stop visit. The two levels therefore observe different state representations at non-aligned decision epochs. TransitDuet handles this asynchrony with a dynamic timetable channel: the upper selects planned-headway adjustments, and the lower executes the resulting timetable through holding.

2.4 Constrained reinforcement learning

Many practical RL deployments must satisfy constraints beyond reward maximisation. Lagrangian relaxation is a widely used approach. Tessler et al. [7] proposed reward-constrained policy optimisation, which augments the policy gradient with dual variables corresponding to cost constraints and performs primal-dual updates to steer the policy toward feasibility. Stooke et al. [28] introduced proportional-integral-derivative (PID)-based Lagrange multiplier updates to reduce oscillation and accelerate convergence. These methods handle soft constraints, for which occasional small violations are tolerable, but do not provide worst-case guarantees.

For settings that demand hard constraint satisfaction, Miryoosefi et al. [9] introduced ApproPO. It formulates the constrained problem as online convex optimisation over occupancy measures and applies a measurement-based projection oracle to enforce feasibility at every iterate. The θ -OGD variant provides a practical implementation that projects policy parameters onto the feasible set using online gradient descent with approximate projections. In the multi-agent domain, Pan et al. [6] proposed RoboDuet for humanoid locomotion. Its upper-body controller and lower-body locomotor exchange cross-advantage terms without centralised control.

TransitDuet combines elements from both lines of work to address the constraint structure of transit operations. Fleet-size feasibility is handled as a soft constraint through the θ -OGD measurement-projection scheme of Miryoosefi et al. [9]. This scheme adaptively reweights an overshoot penalty in the upper-level reward. Per-episode fluctuations under elastic fleet sampling and demand noise can still produce small, bounded overshoot. Headway uniformity is handled as a soft constraint through a Lagrangian penalty with adaptive multiplier updates in the lower-level holding policy. The dynamic timetable channel in Section 4.4 manages cross-level coordination.

3 Problem formulation

We formulate dynamic timetable optimisation and holding control as a bi-level Markov decision process (MDP) for a single bus route with time-varying passenger demand. The upper level makes rolling dispatch-time decisions by selecting a planned-headway adjustment for the next same-direction trip. The lower level executes the resulting dynamic timetable through real-time holding decisions. This section defines the problem variables, action spaces, measurements, and interaction structure used in the experiments. The learning architecture and training mechanisms are specified in Section 4.

3.1 Bus transit system model

We consider a single bidirectional bus route consisting of M stations indexed $\{1, \dots, M\}$. Service is provided according to a timetable that dispatches a total of 264 trips per day, alternating between the two directions. Passenger demand is drawn from time-varying origin–destination (OD) matrices that capture morning, midday, and evening peaks. At any point in time, at most $N_{\text{fleet}}^{\max} = 25$ buses may be concurrently in service; each trip is characterized by a baseline launch time, a planned launch time, an actual launch time, and a travel direction. The planned same-direction dispatch headway is the timetable lever, while per-station holding times constitute the real-time control lever.

3.2 Bi-level MDP formulation

We decompose the joint optimisation into two coupled MDPs that share a common environment but differ in observation granularity, action semantics, and decision frequency.

3.2.1 Upper-level MDP (dynamic timetable planning)

The upper-level policy π_U selects a planned-headway adjustment for the next same-direction dispatch. It is invoked after each successful dispatch event ($M \approx 264$ events per episode, ≈ 132 per direction; one episode is one full service day), so its sampling cadence is event-driven rather than a fixed wall-clock timer.

State.

The upper state encodes system-level summary statistics and the holding-feedback (HoldFB) block that closes the lower-to-upper loop (Section 4.4):

$$s_U = \left[\underbrace{\frac{\text{hour}}{24}, \frac{\text{demand}}{1000}, \frac{\text{fleet_on_route}}{N_{\text{fleet}}^{\max}}, \frac{\text{prev_headway}}{600}}_{\text{system summary, dims 0-4}}, \text{holding_ratio}, \right. \\ \left. \underbrace{\frac{\bar{h}_{\text{hold}}^{\text{up}}}{60}, \frac{\sigma_{\text{hold}}^{\text{up}}}{60}, \frac{\bar{h}_{\text{hold}}^{\text{down}}}{60}}_{\text{HoldFB, dims 5-7}}, \underbrace{\frac{h^{\text{target}}}{600}, \text{dir}, \frac{N_{\text{fleet}}}{20}}_{\text{trip context, dims 8-10}} \right] \in \mathbb{R}^{11}. \quad (1)$$

where each component is normalised to facilitate learning. Dims 5–7 are the rolling holding statistics over the last 10 completed trips (see Section 4.4); the $-$ HoldFB ablation in Section 5.3 zeroes these three dimensions. Dim 10 is the per-episode elastic fleet budget $N_{\text{fleet}} \in [8, 16]$, which the policy must condition on when choosing planned headways under different fleet budgets.

Action.

After trip $k-1$ in a direction is dispatched at actual time τ_{k-1}^{act} , the upper policy selects a scalar planned-headway adjustment:

$$a_U = \Delta h_k \in \{-60, -30, 0, 30, 60\} \text{ s}. \quad (2)$$

The planned headway and planned launch time are

$$h_k^{\text{plan}} = \text{clip}(300 + \Delta h_k, 240, 360), \quad \tau_k^{\text{plan}} = \max(\tau_{k-1}^{\text{act}} + h_k^{\text{plan}}, t + 60). \quad (3)$$

The baseline timetable is retained only as a reference for reporting planned shifts $\tau_k^{\text{plan}} - \tau_k^{\text{base}}$; it no longer constrains dispatch once the dynamic timetable policy is active. If fleet availability or the soft fleet ceiling delays a planned dispatch, the realised lateness is recorded as $\tau_k^{\text{act}} - \tau_k^{\text{plan}}$. The lower controller then receives h_k^{plan} as the trip-specific headway target.

Reward.

The upper reward is computed at the episode level from a measurement vector (defined in Section 3.3). Because each upper action influences the system over an extended horizon through the downstream holding decisions it induces, assigning credit to individual dispatch events is nontrivial; we defer the reward construction to Section 4.

3.2.2 Lower-level MDP (holding control)

The lower-level policy π_L decides how long each bus should dwell beyond its minimum stop time at every station. A decision is triggered at each station arrival for each active bus, producing a high volume of decision epochs throughout the day.

State.

The lower state captures the local context of an individual bus:

$$s_L = [\text{bus_id}, \text{station_id}, \text{hour}, \text{direction}, h_{\text{fwd}}, h_{\text{bwd}}, \hat{d}, \Delta h, a_{\text{last}}, h_k^{\text{plan}}, h_{k-1}^{\text{plan}}, h_{k+1}^{\text{plan}}, \rho_{\text{station}}, v_1, \dots, v_K] \in \mathbb{R}^{13+K}, \quad (4)$$

where h_{fwd} and h_{bwd} denote the forward and backward headways, $\hat{d}$ is the estimated dwell time, Δh is the deviation of the current headway from the planned trip headway, a_{last} is the previous holding action, $h_{k-1:k+1}^{\text{plan}}$ encode adjacent planned dispatch gaps, ρ_{station} is station progress, and $v_1, \dots, v_K$ are recent inter-station speeds that encode local traffic conditions.

Action.

The lower action is a scalar holding time selected from the operational grid used by the final policy:

$$a_L \in \{0, 3, 6, 9, 12, 15, 20, 30\} \text{ seconds}. \quad (5)$$

Reward and cost.

The immediate lower reward is a function of headway deviation from the target set by the upper level. We define the per-step cost as

$$c(s_L, a_L) = \min\left(\frac{(h_{\text{fwd}} - h^{\text{target}})^2}{(h^{\text{target}})^2}, 1\right), \quad (6)$$

which penalises headway irregularity in a scale-invariant manner and is clipped at 1 for numerical stability.

3.3 Constraint specification

The bi-level MDP is subject to two soft constraints, both evaluated through a measurement vector collected at the end of each episode.

Measurement vector.

At the conclusion of a service day, we compute:

$$\mathbf{z} = [\text{avg_wait_min}, \text{peak_fleet}, \text{headway_CV}]. \quad (7)$$

Soft constraint (fleet budget).

The peak number of buses simultaneously on route should respect the per-episode fleet budget $N_{\text{fleet}} \in [N_{\text{fleet}}^{\min}, N_{\text{fleet}}^{\max}]$ sampled at episode start. We enforce this softly via an overshoot² penalty in the upper reward:

$$\text{overshoot}(\mathbf{z}, N_{\text{fleet}}) = \max(0, \text{peak_fleet} - N_{\text{fleet}})^2, \quad (8)$$

weighted by an adaptive coefficient maintained by θ -OGD (see Section 4.5). To prevent unbounded violation under noisy demand, the environment imposes an outer ceiling $N_{\text{fleet}}^{\max} + 3$ buses, which bounds but does not zero out the realised overshoot.

Soft constraint (headway uniformity).

The expected per-step cost must remain below a threshold:

$$\mathbb{E}_{\pi_U, \pi_L}[c(s_L, a_L)] \leq c_{\text{lim}}, \quad (9)$$

which encourages even spacing of buses along the route. Satisfaction of this constraint is enforced via Lagrangian relaxation within the lower-level policy (see Section 4).

3.4 Timescale separation

The principal characteristic of the bi-level formulation is the pronounced separation of timescales between the two decision layers. The upper policy π_U acts at dispatch events, which occur roughly every five minutes on average. The lower policy π_L , by contrast, acts at every station arrival for every active bus, a frequency on the order of once every 30 seconds system-wide when multiple buses are in service. Consequently, hundreds of lower-level decisions may intervene between two consecutive upper-level decisions, and the two layers never act simultaneously.

This asynchrony distinguishes our setting from the semi-Markov options framework [4] and feudal reinforcement learning [29, 25], which assume that (i) the manager and worker share a common state space or a fixed abstraction thereof, and (ii) the manager selects a subgoal that the worker executes to completion before the manager acts again. Neither assumption holds here: the upper state $s_U \in \mathbb{R}^{11}$ summarises fleet-level statistics plus the HoldFB block whereas the lower state $s_L \in \mathbb{R}^{13+K}$ encodes bus-level kinematics, timetable context, and local traffic conditions, and both policies issue decisions driven by environment events rather than by a synchronous clock or a predefined option duration.

More concretely, a single upper action (a planned-headway adjustment Δh_k) determines the planned launch time for the next dispatch while many buses traverse multiple stations under independent lower-level control. The upper policy cannot pause and wait for all induced lower decisions to conclude. Standard HRL methods that propagate value or advantage across levels therefore require modification to handle the fully asynchronous, event-driven interaction between π_U and π_L . We address this challenge through a dynamic-timetable coupling, detailed in Section 4.

Relation to goal-conditioned options.

One can view a dispatched bus as carrying an active goal, namely its planned headway h_k^{plan} , and the lower policy is indeed conditioned on that goal. However, this interpretation alone does not express the part of the problem that TransitDuet optimises. The upper action is not an option assigned to a bus and terminated when that bus reaches a goal. It is a rolling timetable adjustment that determines the planned launch time of a future trip, sometimes before the corresponding vehicle is available, while other buses continue executing previously planned trips. Multiple planned headways are therefore active concurrently, and their effects interact through station-level holding, fleet availability, and dispatch lateness. A conventional goal-conditioned option model with a shared $\pi(a_L | s_L, o_i)$ can represent the lower tracking controller, but it does not by itself specify how future dispatch times are generated, how timetable feasibility is audited, or how credit from passenger wait, onboard delay, and fleet overshoot is assigned back to asynchronous dispatch-time decisions.

4 TransitDuet framework

TransitDuet comprises an upper timetable policy and a lower holding policy linked by a dynamic timetable channel. Figure 1 summarises the architecture; the following subsections specify policy training, cross-level coupling, adaptive fleet-overshoot penalties, and the target-headway-only ablation.

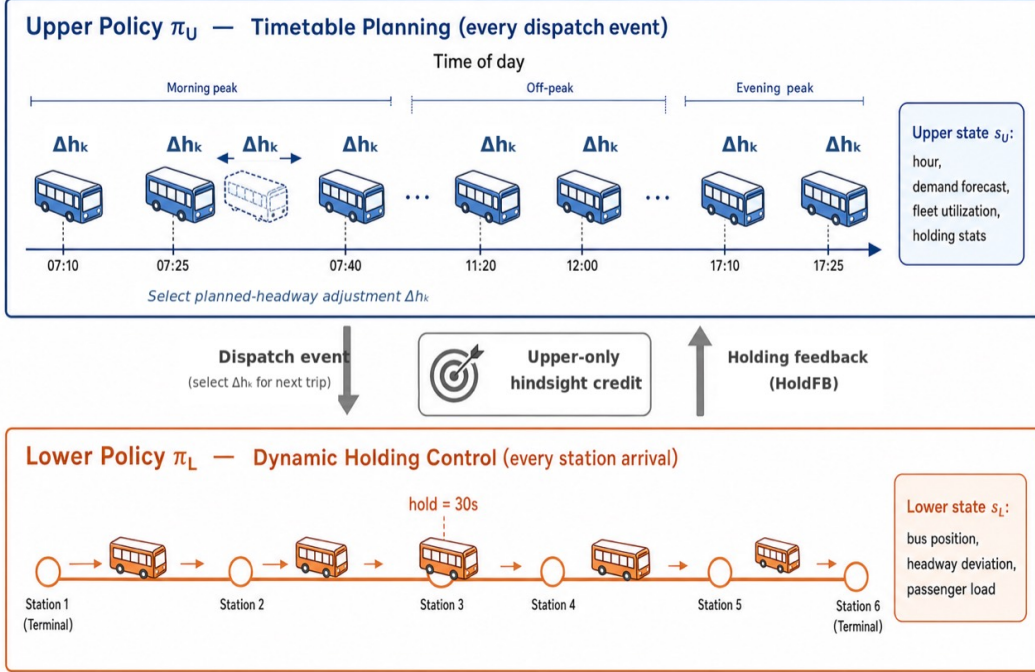


Figure 1 TransitDuet bi-level architecture. The upper policy π_U selects a planned-headway adjustment after each dispatch event; the lower policy π_L selects a holding duration at each station arrival.

4.1 Architecture overview

TransitDuet comprises two independent policy networks operating at different timescales and over different state-action spaces. The upper policy π_U is a lightweight categorical multilayer perceptron (MLP) that selects a planned-headway adjustment for the next same-direction dispatch. The lower policy π_L is a discrete RE-SAC–Lagrangian controller that selects holding durations at every station arrival, with parameters shared across all buses.

A key design choice is that the two networks share no encoder parameters. This differs from the shared-body, dual-head architecture of RoboDuet [6], which couples an upper style policy and a lower tracking policy through a common proprioceptive encoder. Shared encoders are appropriate when the two levels observe the same state at each time step, as in synchronous robot locomotion, but are ill-suited here for two reasons. First, s_U and s_L live in different spaces: s_U encodes route-level demand forecasts and fleet status, whereas s_L encodes per-bus headway deviation, passenger load, and local occupancy. Second, the two levels are triggered asynchronously. The upper policy acts once per dispatch event (roughly once per h^{target} seconds), while the lower policy acts at every station arrival of every active bus, producing hundreds of decisions per upper-level step. Forcing these heterogeneous signals through a shared trunk would create conflicting gradient directions.

The two levels communicate through a dynamic timetable channel (section 4.4). Both networks are trained with off-policy soft actor-critic. The RE-SAC upper reuses per-dispatch transitions from a replay buffer, while the lower uses the same ensemble structure with a Lagrangian dual variable for headway-constraint satisfaction.

4.2 Upper-level timetable policy

The upper policy π_U maps the route-level state s_U to a discrete planned-headway adjustment $\Delta h \in \{-60, -30, 0, 30, 60\}$ s. After a trip is dispatched, this adjustment defines the planned headway and planned launch time of the next same-direction trip according to Equation (3). The lower’s Lagrangian cost (Section 4.3) is then computed against this dispatch-specific planned headway.

Network architecture.

The policy is parameterised as a two-layer MLP with 64 hidden units per layer and rectified linear unit (ReLU) activations, followed by categorical logits over the five timetable actions:

$$\ell_\phi(s_U) = W_3 \text{ReLU}(W_2 \text{ReLU}(W_1 s_U + b_1) + b_2) + b_3. \quad (10)$$

The action distribution is

$$\pi_U(\Delta h \mid s_U) = \text{Categorical}(\text{softmax}(\ell_\phi(s_U))). \quad (11)$$

Off-policy SAC training (RE-SAC upper).

The upper policy is trained as an off-policy soft actor-critic with an ensemble of $K=10$ Q-networks and a lower-confidence-bound (LCB) value in the policy-improvement step. This is the same RE-SAC-style structure [22] used at the lower level (section 4.3), without the Lagrangian. Each dispatch event i contributes a transition $(s_{U,i}, a_{U,i}, R_U^{(i)}, s_{U,i+1}, d_i)$ to a 50k-capacity replay buffer. The per-dispatch reward $R_U^{(i)}$ is the hindsight-credited reward of Equation (21) (section 4.4). The ensemble Q-networks are trained against a single shared mean target to prevent ensemble divergence,

$$y_i = R_U^{(i)} + \gamma(1 - d_i) \left(\frac{1}{K} \sum_{k=1}^K \bar{Q}_k(s_{U,i+1}, \tilde{a}) - \alpha_U \log \pi_U(\tilde{a} \mid s_{U,i+1}) \right). \quad (12)$$

where $\bar{Q}_k$ is the k -th target network and $\tilde{a} \sim \pi_U(\cdot \mid s_{U,i+1})$. The target is clamped to $[-50, 50]$. For the categorical policy, the target value is the probability-weighted expectation over the five action bins. The per-Q mean-squared-error (MSE) loss is augmented with a small ensemble-spread penalty $\beta_{\text{ood}} \cdot \text{std}_k(Q_k)$ that encourages the ensemble to disagree on out-of-distribution actions. The policy is improved by maximizing the LCB value:

$$\mathcal{L}_{\pi_U}(\phi) = \mathbb{E}_{s \sim \mathcal{D}_U, a \sim \pi_U} [\alpha_U \log \pi_U(a \mid s) - (\bar{Q}(s, a) + \beta_{\text{LCB}} \sigma_Q(s, a))]. \quad (13)$$

where $\beta_{\text{LCB}} = -2$, so the LCB value $\bar{Q} + \beta_{\text{LCB}} \sigma_Q$ subtracts twice the ensemble disagreement from the mean. Auto-entropy adjusts α_U toward the categorical target entropy. We use an off-policy SAC upper because dispatches are scarce ($M \approx 264$ per episode), allowing a replay buffer to reuse each transition across gradient updates.

4.3 Lower-level holding control policy

The lower policy π_L selects a non-negative holding action from the grid $\{0, 3, 6, 9, 12, 15, 20, 30\}$ seconds at each station arrival. It is trained with a discrete soft actor-critic controller augmented by a Lagrangian cost term for headway-deviation constraint enforcement [8]. All buses share the same policy parameters, allowing the controller to operate across the fleet from bus-specific input states.

Categorical holding policy.

A two-layer MLP with 64 hidden units per layer and ReLU activations produces categorical logits over the eight holding bins. The discrete formulation avoids asking the actor to learn distinctions smaller than the simulator time resolution and makes the action interface match practical holding commands.

Reward Q-ensemble (RE-SAC, $K=10$).

Rather than the standard twin-Q double-clipped critic of SAC, the lower uses the RE-SAC critic of Zhang and Zheng [22]. It replaces the double clip with an ensemble of $K=10$ independently initialized Q-networks $\{Q_{\omega_k}\}_{k=1}^K$, each a two-layer MLP with 64 hidden units. It also uses a lower-confidence bound rather than the minimum of the two critics for policy improvement. To prevent ensemble divergence, all members are trained against a single shared mean target,

$$y = r + \gamma \left(\frac{1}{K} \sum_{k=1}^K Q_{\omega_k}(s'_L, a'_L) - \alpha \log \pi_L(a'_L \mid s'_L) \right). \quad (14)$$

with $a'_L \sim \pi_L(\cdot \mid s'_L)$, and the per- Q_k loss is augmented with a small ensemble-spread regulariser,

$$\mathcal{L}_Q(\omega_k) = \mathbb{E}_{(s_L, a_L, r, s'_L) \sim \mathcal{D}} [(Q_{\omega_k}(s_L, a_L) - y)^2] + \beta_{\text{ood}} \text{std}_k(Q_{\omega_k}(s_L, a_L)). \quad (15)$$

with $\beta_{\text{ood}} = 0.1$, which encourages members to disagree on out-of-distribution actions. Target parameters $\bar{\omega}_k$ track ω_k via Polyak averaging with $\tau = 0.005$. Policy improvement is performed against the ensemble lower-confidence bound (LCB),

$$\bar{Q}(s, a) = \frac{1}{K} \sum_k Q_{\omega_k}(s, a), \quad \sigma_Q(s, a) = \text{std}_k(Q_{\omega_k}(s, a)). \quad (16)$$

A standard SAC twin-Q critic corresponds to the special case $K=2$ without the LCB; we use the $K=10$ ensemble to represent uncertainty in the lower actor loss below.

Cost Q-network and Lagrangian relaxation.

A separate single-MLP cost critic $Q_\xi^c(s_L, a_L)$ (same hidden width 64) estimates the expected cumulative headway-deviation cost. The lower-level actor loss balances entropy, the ensemble LCB value, and the constraint:

$$\mathcal{L}_{\pi_L}(\psi) = \mathbb{E}_{s_L \sim \mathcal{D}, a_L \sim \pi_L} \left[\alpha \log \pi_L(a_L | s_L) - (\bar{Q}(s_L, a_L) + \beta_{\text{LCB}} \sigma_Q(s_L, a_L)) + \lambda Q_\xi^c(s_L, a_L) \right]. \quad (17)$$

with $\beta_{\text{LCB}} = -2$ (the policy is improved against the lower confidence bound) and $\lambda \geq 0$ the Lagrange multiplier. The dual is updated to satisfy the cost-limit constraint c_{lim} :

$$\lambda \leftarrow \exp \left[\text{clamp}(\log \lambda + \eta_\lambda (\mathbb{E}[Q_\xi^c] - c_{\text{lim}}), -10, 5) \right]. \quad (18)$$

with $\eta_\lambda = 10^{-4}$. Updating in log-space ensures $\lambda > 0$ and the clamp prevents numerical instability. Auto-entropy tuning.

The temperature α is adjusted to maintain a target entropy $\bar{\mathcal{H}} = -\dim(a_L) = -1$:

$$\alpha \leftarrow \min \left(\alpha_{\text{max}}^L, \exp(\log \alpha - \eta_\alpha (\log \pi_L(a_L | s_L) + \bar{\mathcal{H}})) \right), \quad (19)$$

with $\alpha_{\text{max}}^L = 0.08$ to prevent entropy collapse while the timetable and holding policies co-adapt.

4.4 Cross-level coupling: dynamic timetable design

TransitDuet couples the two policies through a dynamic timetable channel. The upper selects a planned-headway adjustment, the environment converts it into a planned launch time, and the lower's Lagrangian cost is computed against the planned headway of the dispatched trip. A target-headway-only channel is included as an ablation variant that changes the lower reference headway without altering planned dispatch times.

Timetable channel: planned dispatch headway.

At each dispatch event, the upper outputs Δh_k and sets h_k^{plan} and τ_k^{plan} via Equation (3). The lower's Lagrangian constraint becomes

$$\mathbb{E}_{\tau \sim \pi_L} \left[|H_{\text{real}}(\tau) - h_k^{\text{plan}}| \right] \leq c_{\text{high}}, \quad (20)$$

where $H_{\text{real}}(\tau)$ is the realised inter-bus headway over the trip lifecycle τ . The upper policy thereby defines an executable rolling timetable rather than only a soft tracking target.

State channel: holding feedback (HoldFB).

The upper state s_U contains a 3-dimensional holding-feedback block at indices 5–7 (Eq. 1; Section 4.7). It carries a rolling 10-trip summary of the lower policy's recent holding decisions: $[\bar{h}_{\text{hold}}^{\text{up}}, \sigma_{\text{hold}}^{\text{up}}, \bar{h}_{\text{hold}}^{\text{down}}]$, normalised to the action range. This channel exposes the intensity of lower-level intervention to the upper policy, allowing it to adapt Δh_k in response to sustained holding.

Reward channel: trip-lifecycle hindsight credit.

At episode end, we compute the realised inter-dispatch gap g_i for each dispatch i from the actual launch times of each bus. We then form a per-dispatch deviation $\Delta_i = |g_i - \bar{g}|/\sigma_g$, where $(\bar{g}, \sigma_g)$

are the per-direction mean and standard deviation. The upper reward at dispatch i combines the episode-level system reward with a per-dispatch hindsight credit:

$$R_U^{(i)} = R_{\text{sys}}(\mathbf{z}) - \alpha_H \cdot \frac{\Delta_i - \bar{\Delta}}{\sigma_\Delta}, \quad \alpha_H = 0.5, \quad (21)$$

where R_{sys} is the belief-weighted multi-objective reward (Eq. 26). The second term recentres the deviation across the episode, so dispatches whose realised lifecycles produced more regular gaps than the episode average earn positive credit. This is upper-only credit assignment; no signal flows back into the lower transitions.

Lower-level reward.

The lower reward is unchanged from Section 4.3: it is the standard environment reward augmented by the Lagrangian headway-deviation cost. Upper coupling redirects the constraint target in Eq. 20 to the current trip’s planned headway. We do not inject upper-level reward signals into lower transitions. Advantage-based auxiliary-reward and target-headway-only channels are retained as comparison rows [30, 31].

Training-stage schedule.

Cross-level coupling is gated by a two-stage schedule. During the first $e_{\text{warm}} = 15$ episodes, the upper policy is inactive and the lower trains against a fixed 300-second rolling timetable. From episode e_{warm} onward, the upper activates and selects planned-headway adjustments. The state and reward channels operate only on the upper side and are inactive during warmup by construction.

4.5 Adaptive fleet-overshoot penalty

The upper policy must respect operational targets on fleet size, average passenger waiting time, and headway regularity. Rather than adding fixed penalty terms to the reward, we use an adaptive scheme inspired by the ApproPO framework [9]. A measurement-driven online gradient descent (θ -OGD) update of the reward-weight vector adjusts relative penalty strengths from realised episode statistics. Unlike full ApproPO, our scheme does not solve a nested best-response oracle and does not provide ApproPO’s feasibility guarantees. It reweights the reward toward the measurements furthest from their targets. In the reported experiments, fleet overshoot remains bounded but is not driven to zero (section 5).

Measurement vector.

At the end of each episode, we compute a three-dimensional measurement vector:

$$\mathbf{z} = [\bar{w}, n_{\text{peak}}, \text{CV}_h] \in \mathbb{R}^3, \quad (22)$$

where $\bar{w}$ is the average passenger waiting time, n_{peak} is the peak fleet size (maximum number of buses simultaneously in service), and CV_h is the coefficient of variation of realised headways.

Weight vector and reward modulation.

A weight vector $\boldsymbol{\theta} \in \mathbb{R}^3$, initialized as $\boldsymbol{\theta}_0 = [-0.5, -0.3, -0.2]$, modulates the upper-level reward:

$$R_U = \boldsymbol{\theta}^\top \mathbf{p}, \quad (23)$$

$$\mathbf{p} = [\bar{w}, (\max(0, n_{\text{peak}} - N_{\text{fleet}}))^2, \text{CV}_h].$$

The quadratic penalty on fleet overshoot $(\max(0, n_{\text{peak}} - N_{\text{fleet}}))^2$ ensures that small violations incur proportionally less cost than large ones, providing a smooth gradient.

Online gradient descent update.

The weight vector is updated by online gradient descent (OGD) on a loss that drives each measurement toward its target:

$$\ell_t = [-\bar{w}_t, -\max(0, n_{\text{peak},t} - N_{\text{fleet}}), -\text{CV}_{h,t}], \quad (24)$$

$$\boldsymbol{\theta}_{t+1} = \Pi_{\mathcal{B}}(\boldsymbol{\theta}_t + \eta_t \ell_t), \quad \eta_t = \frac{\eta_0}{\sqrt{t}}, \quad (25)$$

where $\Pi_{\mathcal{B}}$ projects onto the unit ball $\{\boldsymbol{\theta} : \|\boldsymbol{\theta}\|_2 \leq 1\}$ and η_0 is the base learning rate. The $1/\sqrt{t}$ schedule follows standard online convex optimisation analysis, in which projected OGD on a convex loss has an $O(\sqrt{T})$ regret bound and sublinear cumulative constraint violation. We do not claim the same guarantee for the learned policy. The inner optimisation is a non-convex SAC update on a replay buffer rather than the convex best-response oracle required by ApproPO, and we do not project onto an occupancy-measure feasibility set. We therefore use $\boldsymbol{\theta}$ -OGD as an adaptive penalty whose feasibility properties for the resulting deep-RL policy are empirical rather than provable.

The simplification relative to ApproPO is deliberate. Because the upper policy is trained off-policy with the $\boldsymbol{\theta}$ -modulated reward, repeated SAC gradient updates on the replay buffer act as an approximate best-response step. This eliminates the inner optimisation loop that ApproPO requires, at the cost of the formal guarantee discussed above.

4.6 Training procedure

Algorithm 1 presents the full TransitDuet training loop, which coordinates the upper-warmup schedule with the upper and lower policy updates, the cross-level coupling channels of section 4.4, and the measurement projection.

Lower-only warmup ($e \leq e_{\text{warm}} = 15$).

The upper policy is forced to $\Delta h_k = 0$ for every dispatch, and the lower policy trains in isolation against the baseline timetable. This warmup prevents the upper policy from receiving HoldFB and hindsight-credit signals from an untrained lower policy. We set $e_{\text{warm}} = 15$ episodes.

Coupled training ($e > e_{\text{warm}}$).

The upper policy activates and the timetable channel of section 4.4 engages. After a trip is dispatched, the upper selects a planned-headway adjustment, and the lower Lagrangian cost is computed against that trip-specific planned headway. We do not anneal a coupling coefficient; the timetable channel operates at its configured strength from activation onward.

Computational cost.

The lower policy update dominates computation because of the $K=10$ Q-ensemble and cost-critic gradient steps. Each lower update performs $K+3$ backward passes (K ensemble reward critics, one cost critic, one policy, and one entropy temperature) on graphics-processing-unit (GPU)-batched samples from $\mathcal{D}$. The ensemble is implemented as a single batched forward and backward pass, so wall-clock cost grows sublinearly with K . Each episode contributes $M \approx 264$ upper transitions to a 50k-capacity replay buffer. We run $f_U = 10$ RE-SAC gradient updates per episode on a 64-sample mini-batch. When TPC is enabled for ablation variants, the importance-sampling (IS) weight computation and Polyak target update add $O(M+B)$ scalar/forward-pass operations per episode and are not a bottleneck.

4.7 Auxiliary training components

This subsection collects four secondary mechanisms that interact with the bi-level core.

Holding feedback (HoldFB).

The upper state s_U contains a 3-dimensional holding-feedback block (state indices 5–7) consisting of the lower rolling holding statistics $[\bar{h}_{\text{hold}}^{\text{up}}, \sigma_{\text{hold}}^{\text{up}}, \bar{h}_{\text{hold}}^{\text{down}}]$ over the last 10 completed trips, normalised to the action range. This block reports the intensity of lower-level intervention to the upper policy. The $-\text{HoldFB}$ ablation zeros out these three state dimensions.

Belief-weighted multi-objective reinforcement learning (MORL).

The upper reward is a scalarisation of three measurement components:

$$R_U = -\left[w_{\text{wait}} \frac{\text{wait}}{10} + w_{\text{fleet}} \frac{\text{overshoot}^2}{N_{\text{fleet}}} + w_{\text{cv}} \text{CV}\right], \quad (26)$$

Algorithm 1 TransitDuet training procedure

Require: Episode budget E , upper warmup $e_{\text{warm}} = 15$, lower update frequency f_L , batch size B , hindsight credit weight α_H

- 1: Initialize upper policy π_U , lower policy π_L (RE-SAC ensemble of $K=10$ reward critics + single cost critic), replay buffer $\mathcal{D}$, Lagrange multiplier λ , entropy temperature α , θ -OGD weights, exponential-moving-average (EMA) target upper $\bar{\pi}_U$
- 2: for episode $e = 1, \dots, E$ do
- 3: Reset simulation; sample $N_{\text{fleet}} \sim \text{Uniform}\{8, \dots, 16\}$ if elastic
- 4: $\mathcal{T}_U \leftarrow \emptyset, \mathcal{T}_L \leftarrow \emptyset$
- 5: while episode not done do
- 6: if dispatch event triggered then
- 7: if $e \leq e_{\text{warm}}$ then
- 8: Use fixed 300 s planned headway (upper inactive)
- 9: else
- 10: Sample $\Delta h_k \sim \pi_U(\cdot \mid s_U)$; set h_k^{plan} and τ_k^{plan} via eq. (3)
- 11: Record $(s_{U,i}, a_{U,i})$ in $\mathcal{T}_U$
- 12: end if
- 13: end if
- 14: if station arrival for any bus b then
- 15: Observe s_L ; sample $a_L \sim \pi_L(\cdot \mid s_L)$; apply holding a_L
- 16: Observe r_L, c_L, s'_L ; store $(s_L, a_L, r_L, c_L, s'_L)$ in $\mathcal{D}$ and $\mathcal{T}_L$
- 17: if step mod $f_L = 0$ and $|\mathcal{D}| \geq B$ then
- 18: Sample B transitions from $\mathcal{D}$
- 19: Update π_L critics, policy, λ , α via SAC losses
- 20: Soft-update lower target networks
- 21: end if
- 22: end if
- 23: end while
- 24: Compute episode measurements $\mathbf{z}$, system reward R_{sys} , and per-dispatch hindsight credits via eq. (21)
- 25: Backfill upper transitions in $\mathcal{T}_U$ with $R_U^{(i)}$, push them to the upper replay buffer, and run f_U RE-SAC gradient updates (eq. (12)–(13) on a sampled mini-batch
- 26: Polyak-update the EMA target upper $\bar{\pi}_U$; update θ via OGD on $\mathbf{z}$
- 27: end for

where $\mathbf{w} = (w_{\text{wait}}, w_{\text{fleet}}, w_{\text{cv}})$ is computed from θ -OGD weights modulated by the changepoint-sensitive Bayesian adaptive policy response (CS-BAPR) belief. Under high changepoint probability, mass is shifted from wait/CV to fleet (safety), while during stable windows the converse shift toward wait/CV (quality) is applied. The $-$ MORL ablation replaces $\mathbf{w}$ with the fixed weights $(0.5, 0.25, 0.25)$.

CS-BAPR belief tracker.

A Bayesian online changepoint detector tracks the run length of the upper reward stream and the standard deviation of the lower Q -ensemble. When the changepoint posterior exceeds 0.1, signalling a regime shift in demand or fleet realisation, the lower entropy temperature α is transiently boosted by up to $2\times$ to support exploration in the new regime. The $-$ CS-BAPR ablation disables this boost; the belief weights in MORL also revert to the long-run θ -OGD weights without short-term modulation.

Hindsight credit assignment.

We summarise the hindsight credit term used by the upper reward; the full derivation is given in section 4.4 (Eq. 21). At episode end, we compute the realised inter-dispatch gap g_i per direction, normalise it to a deviation $\Delta_i = |g_i - \bar{g}|/\sigma_g$, and recentre it across the episode so that the average dispatch receives zero credit. The $-$ Hindsight ablation drops this term and uses only the episode-

level reward.

Elastic fleet (Elastic) and demand stochasticity (DemandNoise).

Two environment-side mechanisms further shape training: $N_{\text{fleet}} \sim \text{Uniform}\{8, 9, \dots, 16\}$ is sampled per episode (Elastic), and the per-hour demand multiplier is sampled from $\mathcal{N}(1, 0.15^2)$ clipped to $[0.3, 2.0]$ (DemandNoise). The --Elastic and --DemandNoise ablations fix $N_{\text{fleet}} = 12$ and remove the demand multiplier, respectively, reducing TransitDuet to a deterministic-environment specialist.

4.8 Target-policy correction for ablation variants

TPC is retained for the continuous and target-headway-only variants, where exploratory upper actions can shift the lower’s training distribution away from the deterministic deployment policy. The final rolling-timetable configuration uses categorical timetable actions and does not rely on TPC, but we include the mechanism because it is part of the ablation suite.

Mixed behavior policy.

For continuous upper variants, once the upper policy activates ($e > e_{\text{warm}}$), each dispatch samples δ_t from a mixture $\mu = \epsilon \pi_U(\cdot | s_U) + (1 - \epsilon) \mathcal{N}(\bar{\mu}_U(s_U), \sigma_{\text{tgt}}^2)$, with $\epsilon = 0.25$ exploratory probability and target Gaussian standard deviation $\sigma_{\text{tgt}} = 20\text{s}$ centered at the deterministic mean $\bar{\mu}_U$ of an exponential moving average of the upper policy ($\tau = 0.005$ Polyak rate). The behavior log-prob $\log \mu(\delta_t | s_U)$ and the dispatch metadata $(s_U, \delta_t, \log \mu)$ are stored alongside each lower transition for later use.

Importance-weighted lower SAC.

For every sampled lower transition, the per-sample importance weight

$$w_t = \min\left(\frac{\bar{\pi}_U(\delta_t | s_{U,t})}{\mu(\delta_t | s_{U,t})}, w_{\text{max}}\right)$$

($w_{\text{max}} = 5$) corrects each loss contribution toward the EMA "deployment" upper distribution. The weighted Q, policy, cost-critic, and Lagrangian losses become $L_Q = \mathbb{E}_t[w_t(Q(s_t, a_t) - y_t)^2]$, $L_\pi = \mathbb{E}_t[w_t(\alpha \log \pi_L(a_t | s_t) - Q^{\text{LCB}}(s_t, a_t) + \lambda Q^c(s_t, a_t))]$, and analogously for the cost critic and Lagrangian update. We monitor the effective sample size (ESS), $\text{ESS}/B = (\sum_t w_t)^2 / (B \sum_t w_t^2)$, to detect weight collapse.

Validation-based checkpoint selection.

Every 50 episodes we save (π_U, π_L) to disk. After training, all learned methods are evaluated with the same held-out checkpoint-selection protocol, and the per-seed deployment policy is the checkpoint with the lowest validation composite cost.

Convergence considerations.

The lower-only warmup delays activation of the coupling channels until the lower has trained under the baseline timetable. We use a binary warmup schedule instead of a continuous coupling-coefficient ramp to simplify reproducibility and auditing. The timetable ablations test whether the state-context additions and discrete actions are sufficient without TPC in the main configuration.

5 Experiments

We evaluate TransitDuet on an event-driven bus microsimulation. In the main configuration, the upper policy selects rolling planned-headway adjustments that the simulator converts into planned dispatch times. The lower holding controller tracks the resulting dynamic timetable. A target-headway-only configuration is retained as an ablation.

5.1 Experimental setup

Simulation environment.

The simulator models a single bidirectional bus corridor with 22 stations (11 per direction) and 264 trips per service day. Passenger arrivals follow time-varying origin–destination demand matrices with stochastic per-hour demand multipliers $\mathcal{N}(1, 0.15^2)$ clipped to $[0.3, 2.0]$. Link travel times are log-normal with route stochasticity $\sigma_{\text{route}} = 1.5$ unless otherwise stated. The simulation step is one second; passenger states update every 20 seconds. The upper policy is event-driven and acts after each actual dispatch to select the planned-headway adjustment for the next same-direction trip. The lower policy acts at station arrivals.

Compared methods.

The main method uses a categorical upper policy over five planned-headway adjustments from -60 to $+60$ s and a categorical lower holding policy over eight holding bins from 0 to 30 s. We compare it with the following ablations and baselines:

- Target-headway-only variant: the upper changes the lower reference headway but does not alter planned launch times.
- Wider-action rolling timetable: an ablation with a nine-bin upper action and 60 s maximum holding.
- Continuous rolling timetable: timetable mode with a continuous upper action instead of the categorical timetable action.
- No-context rolling timetable: timetable mode without the lower state additions for last action, neighboring planned gaps, and station phase.
- Fixed-timetable SAC (holding-only SAC): a fixed rolling timetable with the same lower SAC controller, including a 360 s reference and the best fixed headway from $\{300, 330, 360, 390, 420\}$ s.
- Independent assembly: a fixed-timetable lower controller is trained first, frozen, and then paired with a trained timetable upper.
- Classical and RL holding baselines: Daganzo-style holding [10], Xuan-style two-headway holding [11], and a same-simulator advantage-based auxiliary-reward RL proxy motivated by hierarchical RL and recent RL/MARL holding benchmarks [30, 31, 13, 14, 17, 20, 18]. These proxies are not claimed as exact reproductions of the original papers.

Metrics.

In addition to average out-of-vehicle wait, headway CV, fleet overshoot, and the composite cost, we report average holding time, onboard passenger time, total passenger time, and dispatch lateness. We also report the variance of the planned timetable sequence and its correlation with realised headway CV.

The composite cost aggregates the three measurement components into a single evaluation scalar. It is computed as the negated upper-level reward (Eq. 26) under fixed evaluation weights $(w_{\text{wait}}, w_{\text{fleet}}, w_{\text{cv}}) = (0.5, 0.25, 0.25)$, applied uniformly across all methods. Lower composite cost is better.

Training and selection.

Each learned method is trained for 300 episodes per seed with ten seeds: 42, 123, 456, 789, 1001, 1002, 1003, 1004, 1005, and 1006. For each seed we evaluate checkpoints at episodes 49, 99, 149, 199, 249, and 299 with 20 fresh held-out episodes, and select the checkpoint with the lowest mean composite cost when checkpoint evaluation is used. The tables below report the final 30 training episodes per seed because these columns include the newly added holding, passenger-time, and timetable-feasibility diagnostics.

5.2 Main results

Table 1 reports the main comparison from the final 30 training episodes across ten seeds. Because the rolling-timetable implementation changes the action semantics, the target-headway-only configuration is reported as an ablation rather than as the main TransitDuet result.

Table 1 Main comparison under the rolling-timetable protocol. Learned methods report mean values over ten seeds (final 30 training episodes per seed); rule-based methods report ten seeds (30 evaluation episodes per seed). Wait, onboard, and total time are in minutes; holding is in seconds. Lower is better for all metrics.

Method	Wait	CV	Overshoot	Composite	Hold	Onboard	Total time
TransitDuet timetable	6.05	0.484	1.96	1.634	15.59	12.10	18.15
Target-headway only	6.58	0.516	1.93	1.687	8.47	12.72	19.30
Advantage-based RL proxy	7.23	0.524	2.03	1.802	10.22	12.85	20.08
Best fixed timetable SAC (holding-only, 300 s)	6.90	0.444	2.77	1.846	25.69	13.02	19.92
Fixed timetable SAC (holding-only, 360 s)	10.64	0.504	2.28	2.191	29.67	13.66	24.30
Daganzo-style holding proxy	8.86	0.520	2.36	2.040	45.15	13.20	22.06
Xuan-style holding proxy	8.49	0.530	2.47	2.025	48.33	13.34	21.83

TransitDuet achieves the lowest mean composite cost (1.634) in this comparison, 3.2% below the target-headway-only variant (1.687) and 11.5% below the best fixed-timetable SAC baseline (1.846). Its mean total passenger time is 18.15 minutes versus 19.30 and 19.92 minutes for these two comparators.

The table reveals a notable trade-off between headway regularity and fleet utilisation. The best fixed-timetable SAC (300s) achieves the lowest CV (0.444) but incurs the highest fleet overshoot among learned methods (2.77), which drives its composite cost well above TransitDuet’s despite comparable waiting time. This suggests that aggressive holding for regularity can push fleet usage beyond budget constraints.

TransitDuet’s mean holding time (15.59 s) falls between the target-headway-only variant (8.47 s) and the fixed-timetable baselines (25.69–29.67 s). The additional holding relative to the target-headway-only variant translates into a 0.53-minute reduction in mean waiting time and a 1.15-minute reduction in total passenger time, indicating a favourable trade-off. The two classical rules apply substantially longer holding (45–48 s) yet yield higher waiting time and total passenger time, illustrating diminishing returns from excessive holding without timetable adaptation.

5.3 Timetable ablations

Table 2 isolates the design choices in TransitDuet. The target-headway-only row tests adaptive target tuning; the fixed-timetable row isolates whether dynamic planned dispatch times add value beyond the same lower holding controller under a fixed timetable.

Table 2 Timetable ablations. The target-headway-only row does not change planned launch times and is not treated as the main method. Lower is better for each numeric column; Δ Cost is relative to the final rolling-timetable configuration.

Variant	Wait	CV	Cost	Shift variation	Late	Δ Cost
Final rolling timetable	6.05	0.484	1.634	360.3	25.5	–
Target-headway only	6.58	0.516	1.687	–	–	+3.2%
Best fixed timetable SAC	6.90	0.444	1.846	1827.6	38.5	+13.0%
Wider-action rolling timetable	6.18	0.508	1.760	712.9	25.7	+7.7%
Continuous rolling timetable	7.70	0.512	1.856	633.6	12.5	+13.6%
No-context rolling timetable	9.45	0.519	2.048	706.5	18.3	+25.3%
Independent assembly	7.27	0.512	1.929	855.3	34.2	+18.1%

The planned-shift and lateness columns serve as feasibility checks for the timetable claim. A non-zero planned-shift variance confirms that the upper policy actively modifies the planned dispatch

sequence, while dispatch lateness quantifies the delay between planned and actual dispatch caused by vehicle availability or fleet limits.

Among the timetable variants, the wider-action grid (+7.7%) demonstrates that a larger action space does not guarantee improvement; the finer discretisation may introduce additional exploration difficulty without proportional benefit. The continuous variant produces the lowest dispatch lateness (12.5 s) yet the worst composite cost among timetable variants (+13.6%), suggesting that continuous actions complicate policy learning despite allowing smoother adjustments. Removing the lower-state context (planned-dispatch gaps, station phase, and last action) causes the largest degradation (+25.3%), confirming that this contextual information is essential for effective holding under a dynamic timetable.

The independent-assembly result (+18.1%) indicates that joint training contributes to the observed gains: when the lower policy cannot adapt to the rolling timetable during training, performance degrades. This supports describing TransitDuet as a jointly trained framework, although the result does not rule out that alternative coupling strategies could close the gap.

5.4 Timetable variability and realised regularity

For each evaluation episode, we compute the variance of planned dispatch headways and the realised dispatch-headway CV. This diagnostic tests whether variation in the planned sequence is transmitted to realised dispatch regularity.

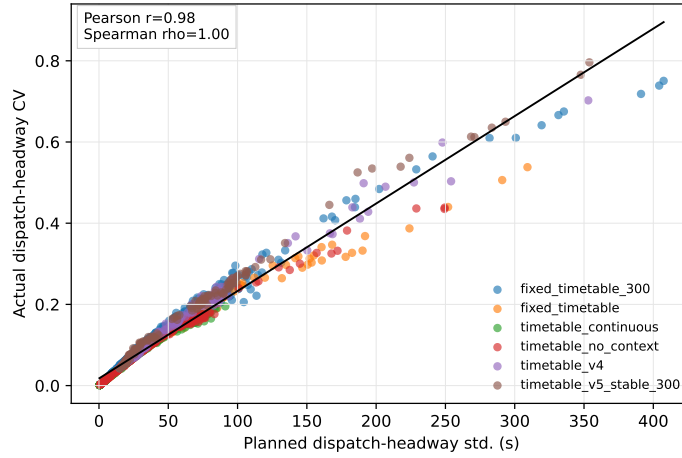


Figure 2 Planned timetable variability versus realised dispatch-headway CV. Each point is one evaluation episode.

Table 3 Correlation between planned timetable variability and realised regularity (300 episode-level observations per method: 10 seeds $\times$ 30 final episodes).

Method	Seeds	Observations	Pearson r	Spearman ρ
Final rolling timetable	10	300	0.994	0.981
Wider-action rolling timetable	10	300	0.990	0.998
Continuous rolling timetable	10	300	0.995	0.999
No-context rolling timetable	10	300	0.989	0.999
Best fixed timetable SAC	10	300	0.982	0.997
Target-headway only	10	300	—	—

The near-perfect correlations (Pearson $r > 0.98$ for all timetable variants) confirm that variation in the planned dispatch sequence is faithfully transmitted to realised dispatch regularity, validating

the timetable channel as an effective control lever. The target-headway-only variant does not alter planned dispatch times and therefore cannot report this correlation.

6 Discussion

Necessity of the timetable channel.

The target-headway-only variant makes the upper decision useful to the lower controller by changing the lower target headway, but it does not alter planned dispatch times. That distinction matters: a target-headway setter can influence realised service through holding, but it does not itself optimise a timetable. In TransitDuet, the upper action defines a planned-headway adjustment that the simulator converts into a planned launch time. This makes the upper decision auditable through planned shifts, planned dispatch headways, and dispatch lateness, while the lower controller executes a dynamic timetable rather than inferring one indirectly.

Relation to conventional HRL.

TransitDuet can be partially interpreted as a goal-conditioned lower controller because each active bus tracks the planned headway associated with its dispatched trip. A conventional option view does not, however, specify a rolling timetable generator. In TransitDuet, several planned headways remain active concurrently, and their effects overlap through shared fleet availability and station-level holding. The challenge is therefore not only that the two levels lack a common clock, but also that multiple timetable goals execute concurrently and credit must be assigned back to asynchronous dispatch-time decisions.

Interpreting joint training.

The independent-assembly ablation is included because joint training is not automatically superior. If a fixed-timetable lower can be trained first and then paired with a timetable upper without loss, the appropriate contribution claim is a jointly trainable framework rather than that joint optimisation is always necessary. Conversely, if the jointly trained version improves total passenger time or dispatch feasibility, the result supports the stronger claim that the lower’s adaptation to the rolling timetable is part of the gain.

Holding-delay trade-off.

Bus holding methods can reduce out-of-vehicle wait and headway CV by increasing onboard delay. The evaluation therefore treats average holding time, onboard passenger time, and total passenger time as first-class metrics. A method that improves wait but worsens total passenger time should not be presented as an unqualified service improvement.

Baseline scope.

The comparison includes timetable-search and bus-holding baselines. Daganzo-style and Xuan-style rules are standard analytical reference points for holding control. Holding-only SAC and same-simulator RL/MARL-style proxies compare TransitDuet with recent RL holding mechanisms without claiming exact reproduction of papers that use different networks, simulators, or demand models. This framing is deliberately conservative: same-simulator proxies test representative mechanisms, not whether TransitDuet supersedes every published implementation.

Limitations.

The current experiments remain single-corridor simulations. Real transit networks introduce interlining, shared terminals, transfer passengers, driver relief constraints, and fleet rebalancing across routes. Scaling the timetable upper to such settings will require factored or graph-structured action representations. The simulator also abstracts away signal priority, incidents, driver heterogeneity, and endogenous passenger abandonment. Finally, the timetable experiments should be read as evidence from a ten-seed evaluation on one calibrated corridor; stronger external-validity claims require additional routes and paired tests on common held-out rollouts.

7 Conclusion

We have presented TransitDuet, a bi-level reinforcement learning framework for dynamic bus timetable optimisation and real-time holding control. The dynamic timetable channel (Section 4.4) links dispatch planning and station holding: the upper policy selects planned-headway adjustments, and the lower policy regulates holding actions against the resulting trip-specific targets. Fleet-size feasibility is maintained through online gradient descent penalty adaptation at the upper level, while Lagrangian relaxation preserves headway regularity at the lower level.

On a calibrated 22-station corridor with stochastic demand and elastic fleet budgets, TransitDuet achieves a mean composite cost 3.2% below the target-headway-only variant and 11.5% below the best fixed-timetable baseline, with a mean total passenger time of 18.15 minutes across ten random seeds. Timetable ablations confirm that the discrete action grid, lower-state context, and joint training each contribute to these gains.

Future work should test TransitDuet on multi-route networks with shared fleets and transfer coordination. Deployment through a calibrated digital twin with domain randomisation could assess robustness to traffic signals and driver heterogeneity. Real-time demand forecasting and passenger-demand elasticity would further connect service quality to ridership.

A Hyperparameter details

Table 4 lists the hyperparameters of the final rolling-timetable configuration used for the main TransitDuet results.

Table 4 Complete hyperparameter specification.

Group	Parameter	Value	Description
Environment	Route speed noise	1.5	Log-normal speed noise σ
	Maximum active buses	25	Hard cap on concurrent buses ($> N_{\text{fleet}}^{\text{max}}$)
	Trips per service day	264	Total trips per service day
	Demand multiplier standard deviation	0.15	Per-hour demand multiplier standard deviation
Upper policy	State dimension	11	System summary (5), HoldFB (3), and trip context (3); see Eq. 1
	Action bins	5 bins	$\Delta h \in \{-60, -30, 0, 30, 60\}$ s
	Minimum shift	-60	Seconds
	Maximum shift	+60	Seconds
	Hidden-layer width	64	MLP hidden-layer width
	Learning rate	3×10^{-4}	Policy and critic learning rate (Adam)
	Mini-batch size	64	Replay-buffer mini-batch
	Updates per episode	10	SAC gradient steps per training episode
	Replay capacity	50,000	Upper transitions stored
	Critic ensemble size K	10	RE-SAC critic ensemble size
	β_{LCB}	-2.0	LCB coefficient in policy improvement
	γ_{U}	0.95	Discount factor (per-dispatch)
	$\alpha_{\text{max}}^{\text{U}}$	0.06	Maximum entropy temperature (auto-tuned)
	$N_{\text{fleet}}^{\text{min}}, N_{\text{fleet}}^{\text{max}}$	8, 16	Elastic fleet sampling range
	N_{fleet} (default)	12	Soft fleet target (overshoot ² penalty)
Lower policy	Hidden-layer width	64	MLP hidden-layer width
	Action bins	8 bins	Holding action $\in \{0, 3, 6, 9, 12, 15, 20, 30\}$ s
	c_{lim}	0.5	Lagrangian cost budget
	Mini-batch size	512	Replay-buffer mini-batch
	Updates per episode	30	SAC gradient steps per training episode
	Learning rate	3×10^{-4}	Critic and policy learning rate
	λ_{lr}	10^{-4}	Lagrangian multiplier learning rate
	γ	0.99	Discount factor
	τ	0.005	Soft target update rate
	Automatic entropy tuning	true	Automatic entropy tuning
	$\alpha_{\text{max}}^{\text{L}}$	0.08	Maximum entropy temperature
	Critic ensemble size K	10	RE-SAC critic ensemble size
	β_{ood}	0.1	Ensemble-spread regulariser weight
	Reward scale	1.0	Reward scaling factor
Coupling	e_{warm}	15 episodes	Lower-only warmup ($\Delta h = 0$)
	α_{H}	0.5	Hindsight-credit weight (Eq. 21)
	TPC ϵ	0.25	Behavior-mixture exploratory probability
	TPC σ_{tgt}	20 s	Target-mode Gaussian standard deviation
	TPC w_{max}	5	Importance-sampling weight clip
	TPC moving-average rate τ	0.005	Polyak rate of exponential-moving-average target upper
	η_{θ}	0.01	ApproPO θ -OGD step size
Training	Training episodes	300	Total training episodes per seed
	Checkpoint interval	50	Episodes
	Lower replay capacity	5×10^5	Lower replay-buffer capacity

B Environment details

The simulation models a single bidirectional bus route calibrated from operational data of an urban bus corridor.

Route topology.

The route comprises M stations connected by inter-station segments. Each direction (upstream and downstream) traverses all M stations in sequence, with terminal turnaround at each end. Station positions and inter-station distances are read from operational records, and segment-level speed profiles are specified per hour of day.

Stochastic travel speed.

Inter-station travel times are driven by a stochastic speed model. At each route-state update, the speed limit on segment $(i, i+1)$ is drawn as

$$v_i = \min(V_{\max}, \max(0, \lfloor \ln \mathcal{LN}(\mu_i(t), \sigma) \rfloor)), \quad (27)$$

where $\mu_i(t)$ is the historical mean log-speed for segment i at hour t , $\sigma = 1.5$ is the log-normal dispersion parameter, and $V_{\max}$ is the segment maximum speed. This produces realistic speed variability that induces headway deviations even under a well-designed timetable.

Passenger generation.

Passenger arrivals follow a Poisson process governed by time-varying origin–destination (OD) matrices. The OD data specifies hourly demand rates between all station pairs. At each simulation second, for every origin station with non-zero demand to a given destination, a Poisson draw with rate $\lambda_{od}/3600$ determines the number of new passengers. Each passenger records an appearance time, an origin, and a destination; waiting time is measured from appearance to boarding.

Bus operations.

Each bus maintains a state machine with four phases:

1. Holding: the bus is held at a station for the duration specified by the lower-level policy (0–30 s in the final configuration).
2. Dwelling: passengers alight and board. Boarding proceeds until the bus reaches capacity ($C = 50$ passengers) or no more waiting passengers have a valid destination downstream.
3. Travel: the bus moves toward the next station at the current stochastic speed. Acceleration (3 m/s^2) and deceleration (5 m/s^2) are modeled at departure and arrival.
4. Waiting for action: upon arriving at a station, the bus waits for the next holding action from the lower-level policy.

Forward and backward headways are computed with respect to the preceding and following buses in the same direction, and the headway deviation from the target set by the upper policy defines the per-step cost (Equation (6)).

Timetable structure.

A service day consists of 264 trips alternating between the two directions, truncated from a full operational schedule to keep simulation tractable. At most 25 buses may be concurrently active. In the main configuration, the upper policy selects a planned-headway adjustment for the next same-direction trip. The simulator converts the resulting planned headway into a planned launch time and dispatches the trip once the planned time and vehicle-availability constraints are satisfied. The baseline timetable is retained only as a reference for reporting planned shifts and deviations.

Episode termination.

An episode ends when all 264 trips have been launched and every bus has completed its trip (i.e., no bus remains on-route). At this point the measurement vector $\mathbf{z}$ (Equation (7)) is computed for the ApproPO θ -OGD update.

C Network architectures

All networks use rectified linear unit (ReLU) activations and are trained with Adam. Weight initialization follows PyTorch defaults unless otherwise noted.

Upper policy π_U (categorical timetable action).

A two-layer multilayer perceptron (MLP) maps the 11-dimensional upper state (5 system summary dimensions + 3 holding-feedback dimensions + 3 trip-context dimensions; eq. (1)) to logits over five planned-headway adjustments $\Delta h \in \{-60, -30, 0, 30, 60\}$ s:

Layer	Shape	Activation
Linear	11 $\rightarrow$ 64	ReLU
Linear	64 $\rightarrow$ 64	ReLU
Logit head	64 $\rightarrow$ 5	Softmax for sampling

The selected bin defines $h^{\text{plan}} = \text{clip}(300 + \Delta h, 240, 360)$ s. The simulator then sets the next same-direction planned launch time to the maximum of the previous actual dispatch plus h^{plan} and the current time plus a 60s notice window. The continuous upper policy remains available as the continuous rolling-timetable ablation.

Lower policy π_L (categorical holding action).

A two-layer MLP maps the lower state to logits over eight holding bins $\{0, 3, 6, 9, 12, 15, 20, 30\}$ s:

Layer	Shape	Activation
Linear	$(13+K) \rightarrow 64$	ReLU
Linear	64 $\rightarrow$ 64	ReLU
Logit head	64 $\rightarrow$ 8	Softmax for sampling

The additional five lower-state dimensions are the previous holding action, the current trip’s planned headway, the previous and next planned same-direction gaps, and the station phase. The same network is shared across all buses, with bus-specific behavior arising from heterogeneous input states.

Reward Q-ensemble $\{Q_k\}_{k=1}^K$ (RE-SAC, $K=10$).

The lower replaces the standard twin-Q SAC critic with an ensemble of $K=10$ independently initialized Q-networks. The policy is improved against a lower-confidence-bound (LCB) value $\bar{Q} + \beta_{\text{LCB}}\sigma_Q$ with $\beta_{\text{LCB}} = -2$, computed across the ensemble at every gradient step. Each ensemble member shares the same MLP shape:

Layer	Shape	Activation
Linear	$(13+K+1) \rightarrow 64$	ReLU
Linear	64 $\rightarrow$ 64	ReLU
Linear	64 $\rightarrow$ 1	—

Each Q_k takes the concatenation of the lower state and scalar action as input. All members share a common mean target $(1/K)\sum_k \bar{Q}_k$ during temporal-difference learning to prevent ensemble divergence. The per-Q mean-squared-error (MSE) loss is augmented with a small ensemble-spread penalty $\beta_{\text{ood}}\text{std}_k(Q_k)$ ($\beta_{\text{ood}}=0.1$) that encourages disagreement on out-of-distribution actions. All ensemble members have exponential-moving-average target networks with Polyak rate $\tau = 0.005$. We use a Q-ensemble rather than a twin Q to construct lower-confidence bounds; the twin-Q variant in standard SAC corresponds to $K=2$ without the LCB.

Cost Q-network Q_c .

A single MLP with the same MLP shape as one ensemble member estimates the expected cumulative cost (headway deviation) under the current policy:

Layer	Shape	Activation
Linear	$(13+K+1) \rightarrow 64$	ReLU
Linear	64 $\rightarrow$ 64	ReLU
Linear	64 $\rightarrow$ 1	—

The cost critic is trained with the same target-network mechanism and soft update rate as the reward critics. Its output enters the policy loss through the Lagrangian multiplier λ : the policy minimises $\alpha \log \pi - Q + \lambda Q_c$.

D Auxiliary hindsight credit: derivation and contrast with cross-advantage

The main cross-level coupling in TransitDuet is the rolling timetable channel of section 4.4. This appendix derives the auxiliary hindsight credit on the upper reward (Eq. 21 in section 4.4), an upper-only signal for per-dispatch credit assignment. It also contrasts this mechanism with a literal asynchronous extension of the cross-advantage objective of Pan et al. [6].

D.1 Synchronous cross-advantage (RoboDuet, background)

In the synchronous setting of Pan et al. [6], two agents act at every time step t on a shared state s_t . The cross-advantage augmented objective is

$$\mathcal{L}_{\text{upper}}^{\text{sync}} = -\mathbb{E}_t[(A_U(s_t, a_t^U) + \beta A_L(s_t, a_t^L)) \nabla \log \pi_U(a_t^U | s_t)], \quad (28)$$

which is well-defined because both advantages are evaluated at the same shared state-action pair.

D.2 The asynchrony challenge

In TransitDuet, π_U acts at dispatch events $\{t_1^U, \dots, t_N^U\}$ while π_L acts at station arrivals $\{t_1^L, \dots, t_M^L\}$ with $M \gg N$, on disjoint event sets and different state spaces. There is no single time step at which A_U and A_L are jointly defined on a shared state, so the direct sum in Eq. (28) is undefined.

D.3 Trip-lifecycle alignment and the realised gap

We resolve this through trip-lifecycle alignment. Each upper action $a_{U,i}$ after dispatch $i-1$ selects the planned-headway adjustment that determines the planned launch time and planned headway for trip i . The realised inter-dispatch gap is the elapsed time between consecutive bus departures after vehicle availability and holding interactions are applied:

$$g_i = t_{\text{actual},i+1} - t_{\text{actual},i}, \quad (29)$$

computed per direction so that buses operating on opposite legs do not interleave. The per-direction mean and standard deviation of $\{g_i\}$ define the deviation

$$\Delta_i = \frac{|g_i - \bar{g}|}{\sigma_g}. \quad (30)$$

D.4 Hindsight-credited upper reward

The upper reward at dispatch i is the sum of the episode-level system reward (a belief-weighted MORL scalarisation of wait, fleet overshoot, and headway CV; Eq. 26) and a per-dispatch hindsight credit derived from Δ_i :

$$R_U^{(i)} = R_{\text{sys}}(\mathbf{z}) - \alpha_H \cdot \frac{\Delta_i - \bar{\Delta}}{\sigma_{\Delta}}, \quad \alpha_H = 0.5. \quad (31)$$

The credit is recentered per episode so that dispatches with above-average gap deviation receive a negative credit and dispatches with below-average deviation receive a positive one. The system reward is shared across all dispatches in an episode.

D.5 Why we did not use literal cross-advantage

A literal asynchronous extension of Eq. (28) would aggregate per-trip lower advantages into the upper objective:

$$\tilde{R}_U^{\text{lit},(i)} = R_U^{(i)} + \beta \cdot \frac{1}{|\mathcal{J}(i)|} \sum_{j \in \mathcal{J}(i)} A_L(s_{L,j}, a_{L,j}), \quad (32)$$

where $\mathcal{J}(i)$ is the set of lower-level transitions during trip i . An early prototype of this objective was unstable in our runs. The per-step lower advantage and the per-dispatch upper advantage differed in scale by 1–2 orders of magnitude across a trip lifecycle. A single scalar β therefore either suppressed the cross-level signal or destabilised lower training. In addition, the lower advantage estimator uses an LCB-discounted ensemble Q (section 4.3), which is a nonlinear function of the policy parameters. The simple unbiasedness argument for the synchronous setting therefore does not transfer directly. The hindsight-credit formulation in Eq. (31) instead injects the per-trip execution outcome (gap regularity) into upper credit assignment without requiring lower-advantage backflow.

D.6 What changes for our off-policy SAC upper

The TransitDuet upper is trained as RE-SAC (section 4.2, Eqs. 12–13), not as REINFORCE. Each per-dispatch transition is pushed to a replay buffer, and the policy and critic are updated by repeated mini-batch SAC gradient steps. The REINFORCE-based unbiasedness argument for a synchronous on-policy estimator is therefore not the appropriate analytical lens for this setup. The lower is also updated between upper SAC gradient steps, so the upper training distribution is non-stationary. The timetable ablations evaluate whether joint training is necessary or whether a frozen independently trained lower is sufficient. The upper ensemble-LCB Q value penalises epistemic uncertainty in its off-policy training objective.

References

- [1] Gkiotsalitis K, Cats O. At-stop control measures in public transport: literature review and research agenda. *Transp Res Part E: Logist Transp Rev.* 2021;145:102176.
- [2] Wang J, Sun L. Dynamic holding control to avoid bus bunching: a multi-agent deep reinforcement learning framework. *Transp Res Part C: Emerg Technol.* 2020;116:102661.
- [3] Zhang Y, Zheng L, Zhang Q, Tang H. Single agent robust deep reinforcement learning for bus fleet control. *Transp Saf Environ.* 2026;00(0):tdag005.
- [4] Sutton RS, Precup D, Singh S. Between MDPs and semi-MDPs: a framework for temporal abstraction in reinforcement learning. *Artif Intell.* 1999;112(1–2):181-211.
- [5] Nachum O, Gu S, Lee H, Levine S. Data-efficient hierarchical reinforcement learning. In: *Advances in Neural Information Processing Systems.* vol. 31; 2018. p. 3303-13.
- [6] Pan G, Ben Q, Yuan Z, Jiang G, Ji Y, Li S, et al. RoboDuet: learning a cooperative policy for whole-body legged loco-manipulation. *IEEE Robot Autom Lett.* 2025;10(5):4564-71.
- [7] Tessler C, Mankowitz DJ, Mannor S. Reward constrained policy optimization. In: *Proceedings of the 7th International Conference on Learning Representations*; 2019. .
- [8] Haarnoja T, Zhou A, Abbeel P, Levine S. Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *Proceedings of the 35th International Conference on Machine Learning.* vol. 80 of *Proceedings of Machine Learning Research.* PMLR; 2018. p. 1861-70.
- [9] Miryoosefi S, Brantley K, Daumé III H, Dudík M, Schapire RE. Reinforcement learning with convex constraints. In: *Advances in Neural Information Processing Systems.* vol. 32; 2019. p. 14093-102.
- [10] Daganzo CF. A headway-based approach to eliminate bus bunching: systematic analysis and comparisons. *Transp Res Part B: Methodol.* 2009;43(10):913-21.
- [11] Xuan Y, Argote J, Daganzo CF. Dynamic bus holding strategies for schedule reliability: optimal linear control and performance analysis. *Transp Res Part B: Methodol.* 2011;45(10):1831-45.
- [12] Alesiani F, Gkiotsalitis K. Reinforcement learning-based bus holding for high-frequency services. In: *Proceedings of the 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*; 2018. p. 3162-8.
- [13] Wang J, Sun L. Reducing bus bunching with asynchronous multi-agent reinforcement learning. In: *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*; 2021. p. 426-33.
- [14] Wang J, Sun L. Robust dynamic bus control: a distributional multi-agent reinforcement learning approach. *IEEE Trans Intell Transp Syst.* 2023;24(4):4075-88.
- [15] Wang J, Sun L. Multi-objective multi-agent deep reinforcement learning to reduce bus bunching for multilane services with a shared corridor. *Transp Res Part C: Emerg Technol.* 2023;155:104309.
- [16] Rodriguez J, Koutsopoulos HN, Wang S, Zhao J. Cooperative bus holding and stop-skipping: a deep reinforcement learning framework. *Transp Res Part C: Emerg Technol.* 2023;155:104308.
- [17] Yu M, Yang T, Li C, Jin Y, Xu Y. Mitigating bus bunching via hierarchical multi-agent reinforcement learning. *IEEE Trans Intell Transp Syst.* 2024;25(8):9675-92.

- [18] Li C, Shen M, Hu Q, Zhen L, Xiao F. Dynamic bus holding control for multi-line busy bus corridors: mitigating bus queues and improving headway regularity via graph-aware deep reinforcement learning. *Transp Res Part E: Logist Transp Rev.* 2026;207:104603.
- [19] He SX, He JJ, Liang SD, Dong JQ, Yuan PC. A dynamic holding approach to stabilizing a bus line based on the Q-learning algorithm with multistage look-ahead. *Transp Sci.* 2022;56(1):31-51.
- [20] Xu Z, Wang C, Shen M, Li C, Liu X. Reinforcement learning for bus bunching mitigation: a systematic evaluation of configurations and performances. *IEEE Trans Intell Transp Syst.* 2025;26(7):10443-55.
- [21] Yu J, Wang Y, Ma W. Large language model-enhanced reinforcement learning for generic bus holding control strategies. *Transp Res Part E: Logist Transp Rev.* 2025;200:104142.
- [22] Zhang Y, Zheng L. RE-SAC: disentangling aleatoric and epistemic risks in bus fleet control: a stable and robust ensemble DRL approach. *arXiv preprint.* 2026.
- [23] Ibarra-Rojas OJ, Delgado F, Giesen R, Muñoz JC. Planning, operation, and control of bus transport systems: a literature review. *Transp Res Part B: Methodol.* 2015;77:38-75.
- [24] Cats O, Glück S. Frequency and vehicle capacity determination using a dynamic transit assignment model. *Transp Res Record: J Transp Res Board.* 2019;2673(3):574-85.
- [25] Vezhnevets AS, Osindero S, Schaul T, Heess N, Jaderberg M, Silver D, et al. FeUdal networks for hierarchical reinforcement learning. In: *Proceedings of the 34th International Conference on Machine Learning.* vol. 70 of *Proceedings of Machine Learning Research.* PMLR; 2017. p. 3540-9.
- [26] Lee J, Lee BJ, Kim KE. Reinforcement learning for control with multiple frequencies. In: *Advances in Neural Information Processing Systems.* vol. 33; 2020. p. 3254-64.
- [27] Metelli AM, Mazzolini F, Bisi L, Sabbioni L, Restelli M. Control frequency adaptation via action persistence in batch reinforcement learning. In: *Proceedings of the 37th International Conference on Machine Learning.* vol. 119 of *Proceedings of Machine Learning Research.* PMLR; 2020. p. 6862-73.
- [28] Stooke A, Achiam J, Abbeel P. Responsive safety in reinforcement learning by PID Lagrangian methods. In: *Proceedings of the 37th International Conference on Machine Learning.* vol. 119 of *Proceedings of Machine Learning Research.* PMLR; 2020. p. 9133-43.
- [29] Dayan P, Hinton GE. Feudal reinforcement learning. In: *Advances in Neural Information Processing Systems.* vol. 5; 1992. p. 271-8.
- [30] Li S, Wang R, Tang M, Zhang C. Hierarchical reinforcement learning with advantage-based auxiliary rewards. In: *Advances in Neural Information Processing Systems.* vol. 32; 2019. p. 1409-19.
- [31] Singh U, Suttle WA, Sadler BM, Namboodiri VP, Bedi AS. PIPER: primitive-informed preference-based hierarchical reinforcement learning via hindsight relabeling. *arXiv preprint.* 2024.